PEP 604: Allow writing union types as ``X | Y``

Status: Final · Created: 28-Aug-2019 · Public domain / CC0

Abstract

This PEP proposes overloading the | operator on types to allow writing Union[X, Y] as X | Y, and allows it to appear in isinstance and issubclass calls.

Motivation

484 and 526 propose a generic syntax to add typing to variables, parameters and function returns. 585 proposes to expose parameters to generics at runtime. Mypy accepts a syntax which looks like::

```
annotation: name_type
name_type: NAME (args)?
args: '[' paramslist ']'
paramslist: annotation (',' annotation)* [',']
```

- To describe a disjunction (union type), the user must use Union[X, Y].

The verbosity of this syntax does not help with type adoption.

Proposal

Inspired by Scala and Pike , this proposal adds operator type.__or__(). With this new operator, it is possible to write int | str instead of Union[int, str]. In addition to annotations, the result of this expression would then be valid in isinstance() and issubclass()::

```
isinstance(5, int | str)
issubclass(bool, int | float)
```

We will also be able to write t | None or None | t instead of Optional[t]::

```
isinstance(None, int | None)
isinstance(42, None | int)
```

Specification

The new union syntax should be accepted for function, variable and parameter annotations.

Simplified Syntax

::

```
# Instead of
# def f(list: List[Union[int, str]], param: Optional[int]) -> Union[float, str]
def f(list: List[int | str], param: int | None) -> float | str:
    pass
f([1, "abc"], None)
# Instead of typing.List[typing.Union[str, int]]
typing.List[str | int]
list[str | int]
# Instead of typing.Dict[str, typing.Union[int, float]]
typing.Dict[str, int | float]
dict[str, int | float]
```

The existing typing.Union and | syntax should be equivalent.

::

int | str == typing.Union[int, str]

```
typing.Union[int, int] == int
```

```
int | int == int
```

The order of the items in the Union should not matter for equality.

::

```
(int | str) == (str | int)
(int | str | float) == typing.Union[str, float, int]
```

Optional values should be equivalent to the new union syntax

::

None | t == typing.Optional[t]

A new Union.__repr__() method should be implemented.

::

```
str(int | list[str])
# int | list[str]
str(int | int)
# int
```

isinstance and issubclass

The new syntax should be accepted for calls to isinstance and issubclass as long as the Union items are valid arguments to isinstance and issubclass themselves.

::

```
# valid
isinstance("", int | str)
# invalid
isinstance(2, list[int]) # TypeError: isinstance() argument 2 cannot be a parameterized generic
isinstance(1, int | list[int])
# valid
issubclass(bool, int | float)
# invalid
issubclass(bool, bool | list[int])
```

Incompatible changes

In some situations, some exceptions will not be raised as expected.

If a metaclass implements the __or__ operator, it will override this::

```
>>> class M(type):
...     def __or__(self, other): return "Hello"
...
>>> class C(metaclass=M): pass
...
>>> C | int
'Hello'
>>> int | C
typing.Union[int, __main__.C]
>>> Union[C, int]
typing.Union[__main__.C, int]
```

Objections and responses

For more details about discussions, see links below:

- Discussion in python-ideas - Discussion in typing-sig

1. Add a new operator for Union[type1, type2]?

PROS:

- This syntax can be more readable, and is similar to other languages (Scala, ...) - At runtime, int|str might return a simple object in 3.10, rather than everything that you'd need to grab from importing typing

CONS:

- Adding this operator introduces a dependency between typing and builtins - Breaks the backport (in that typing can easily be backported but core types can't) - If Python itself doesn't have to be changed, we'd still need to implement it in mypy, Pyre, PyCharm, Pytype, and who knows what else (it's a minor change see "Reference Implementation")

2. Change only PEP 484 (Type hints) to accept the syntax type1 | type2 ?

563 (Postponed Evaluation of Annotations) is enough to accept this proposition, if we accept to not be compatible with the dynamic evaluation of annotations (eval()).

::

```
>>> from __future__ import annotations
>>> def foo() -> int | str: pass
...
>>> eval(foo.__annotations__['return'])
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<string>", line 1, in <module>
TypeError: unsupported operand type(s) for |: 'type' and 'type'
```

3. Extend isinstance() and issubclass() to accept Union ?

::

```
isinstance(x, str | int) ==> "is x an instance of str or int"
```

PROS:

- If they were permitted, then instance checking could use an extremely clean-looking notation

CONS:

- Must migrate all of the typing module in builtin

Reference Implementation

A new built-in Union type must be implemented to hold the return value of t1 | t2, and it must be supported by isinstance() and issubclass(). This type can be placed in the types module. Interoperability between types.Union and typing.Union must be provided.

Once the Python language is extended, mypy and other type checkers will need to be updated to accept this new syntax.

- A proposed implementation for cpython is here. - A proposed implementation for mypy is here.

References

Copyright

This document is placed in the public domain or under the CC0-1.0-Universal license, whichever is more permissive.